HDF5 Data Stores

Concepts and NASDAQ ITCH Application

IEOR E4576 — Data-Driven Models in Finance — Fall 2026 — Week 5

Contents

1 What is HDF5? 1

2 Fundamental Objects in an HDF5 File 1

3 Key Features 2

4 h5py: The Direct Python Interface 2

5 PyTables and pandas HDFStore 3

6 Minimal Working Example 4

7 NASDAQ ITCH 5.0 Schema 4

8 Categorical Encoding Reference 5

1 What is HDF5?

Hierarchical Data Format 5 (HDF5) is an open binary file format for storing and organising large numerical datasets efficiently. Originally developed at NCSA for scientific simulation output, it is now managed by The HDF Group and is standard in finance, genomics, and high-energy physics.

2 Fundamental Objects in an HDF5 File

An HDF5 file is built from two primary objects, **groups** and **datasets**, plus **attributes** that attach metadata to either. Groups and datasets are connected by **links**, which give every object its path name.

Object	Analogy	What it holds	h5py class
File	A disk volume	The container. Every file has exactly one *root group*, `/`.	`h5py.File`
Group	Directory	Named links to other groups and datasets. Groups nest to any depth.	`h5py.Group`
Dataset	File / array on disk	An n-dimensional array of elements of a single type — the actual data.	`h5py.Dataset`
Attribute	File properties / tags	A small named value (scalar, string, short array) attached to a group or dataset.	`obj.attrs`
Link	Directory entry	The name under which an object appears in a group. *Hard* links point to an object in the same file, *soft* links to a path, *external* links to an object in another file.	`h5py.SoftLink`, `h5py.ExternalLink`

Every dataset is in turn described by three properties, all stored in the file:

Datatype
The type of one element: `float64`, `int32`, a fixed-length string, or a *compound* type (a record with named fields, like a NumPy structured array or one row of a table).

Dataspace
The shape, e.g. `(3,)` or `(1000, 5)`, and the maximum shape; a dimension with unlimited maximum (`maxshape=(None,)`) can be extended later.

Storage layout
Contiguous (one block on disk) or *chunked* (fixed-size blocks, required for compression and for resizable datasets).

The h5py example later in these notes produces this tree:

```
/                        group (root)      attrs: date
‘-- equities             group
    ‘-- AAPL             group
        |-- prices       dataset  float64 (3,)   attrs: units, source
        ‘-- volume       dataset  int32   (3,)   attrs: units
```

`pandas.HDFStore` uses the same objects: each key such as `/A` is a *group*; the DataFrame rows live in a single *dataset* named `table` inside it, whose compound datatype has one field per column; and the indexes on `data_columns` are stored as further groups and datasets alongside it.

3 Key Features

Filesystem inside a file
Groups play the role of directories and datasets the role of files. Groups nest arbitrarily; every object is addressed by a POSIX-style path, e.g. `/equities/AAPL/trades`.

Self-describing schema
Every group and dataset carries typed *attributes* (key–value metadata). The schema travels with the data, so no external catalogue is needed.

Chunked storage
Datasets can be split into fixed-size *chunks* stored independently. Only the relevant chunks are read on a partial access, giving column-store-like I/O for row-oriented scientific arrays.

Transparent compression
GZIP, LZF, and Blosc compressors can be applied per dataset. Compression is handled by the HDF5 library; the Python read/write API is unchanged.

4 h5py: The Direct Python Interface

h5py provides a thin, Pythonic wrapper around the HDF5 C library. Groups and datasets behave like Python dicts; datasets behave like NumPy array proxies. Use h5py when you need full control over the file structure, custom datatypes, or HDF5 features not exposed by PyTables.

Opening files
`h5py.File(path, mode)` accepts modes `'w'` (create/overwrite), `'r'` (read-only), `'r+'` (read/write, must exist), and `'a'` (read/write, create if absent). Always use it as a context manager so the file is flushed and closed on exit.

Groups and datasets
`f.create_group('/equities/AAPL')` creates nested groups in one call. `f.create_dataset('prices', data=arr)` writes a NumPy array. Both support dict-style access: `f['equities'].keys()`, `'prices' in f`.

Lazy array access and slicing
`f['prices']` returns a dataset *proxy* — no data is transferred yet. Data arrives only on explicit indexing: `f['prices'][:]`, `f['prices'][0:1000]`, `f['prices'][::10]`. Only the HDF5 chunks that overlap the slice are read from disk.

Chunking and compression
Pass `chunks=(10000,)` and `compression='gzip'` to `create_dataset`. Each chunk is stored and compressed independently, so a partial read decompresses only the touched blocks. Blosc (`compression='lzf'`) is faster than gzip at modest ratios.

Attributes
`f['prices'].attrs['units'] = 'USD cents'` attaches typed scalar or array metadata to any group or dataset. Attributes are stored in the HDF5 file itself and are readable without loading the dataset.

Inspecting a file
`f.visit(print)` recursively lists every path in the file. `f.visititems(fn)` passes both the path string and the object to a callback, useful for auditing an unfamiliar HDF5 file.

```python
import h5py, numpy as np

prices = np.array([150.10, 150.12, 150.09], dtype='f8')
volume = np.array([100,    200,    150   ], dtype='i4')

# --- Write ---
with h5py.File('example_h5py.h5', 'w') as f:
    grp = f.create_group('equities/AAPL')          # creates parent groups too

    ds_prices = grp.create_dataset('prices', data=prices,
                                   chunks=(1024,),
                                   compression='gzip', compression_opts=4)
    ds_prices.attrs['units']  = 'USD'              # metadata on the dataset
    ds_prices.attrs['source'] = 'NASDAQ ITCH 5.0'

    ds_volume = grp.create_dataset('volume', data=volume,
                                   chunks=(1024,),
                                   compression='gzip', compression_opts=4)
    ds_volume.attrs['units'] = 'shares'

    f.attrs['date'] = '2019-10-30'                 # metadata on the file

# --- Inspect ---
with h5py.File('example_h5py.h5', 'r') as f:
    f.visit(print)                                 # prints: equities/AAPL/prices
    print(f['equities/AAPL/prices'].attrs['units'])#         USD

# --- Read: full array into NumPy ---
with h5py.File('example_h5py.h5', 'r') as f:
    p = f['equities/AAPL/prices'][:]               # shape (3,), dtype float64

# --- Read: partial slice (only touched chunks hit disk) ---
with h5py.File('example_h5py.h5', 'r') as f:
    first_two = f['equities/AAPL/prices'][0:2]
```

h5py, PyTables, or `HDFStore`? h5py gives maximum flexibility and direct HDF5 access, but has no query engine — filtering is done in Python after loading. PyTables adds typed `Table` objects with in-kernel (C-level) `where` queries; `pandas.HDFStore` wraps PyTables further so DataFrames go in and DataFrames come out with minimal boilerplate. For raw numerical arrays or custom schemas use h5py; for tabular financial data with row-level queries use `HDFStore`.

5 PyTables and pandas HDFStore

PyTables (`tables`) is a higher-level Python layer on top of HDF5 that adds relational-style *Table* objects with typed columns, column indexes, and in-kernel (server-side) queries. `pandas.HDFStore` wraps PyTables to read and write DataFrames.

`fixed` format
Stores the DataFrame as a single monolithic array. Very fast reads, but the entire dataset must be loaded at once — no partial or query-based access. Suitable for small, immutable lookup tables.

`table` format (`format='t'`)
Stores the DataFrame as a PyTables `Table`: each column is a typed field, rows can be appended incrementally with `store.append`, and subsets can be retrieved server-side with `store.select(key, where='col > val')`. This is the right choice for tick data.

`data_columns`
Columns listed here receive a PyTables index, making them usable in `where` clauses. Only indexed columns can appear in a `select` filter. Choosing too many `data_columns` increases write overhead.

`min_itemsize`
String (object-dtype) columns require a declared maximum byte length at table creation time. Passing `{'stock': 8}` as `min_itemsize` reserves 8 bytes per row for the `stock` field. Columns encoded to integers do *not* need an entry.

6 Minimal Working Example

```
import pandas as pd

# --- Build a small trade DataFrame ---
df = pd.DataFrame({
    'timestamp': pd.to_timedelta(['09:30:00', '09:30:01', '09:30:02']),
    'price'    : [150.10, 150.12, 150.09],
    'volume'   : [100, 200, 150],
    'side'     : [1, -1, 1],       # 1 = buy, -1 = sell (pre-encoded)
    'stock'    : ['AAPL', 'AAPL', 'AAPL'],
})

# --- Write in table format ---
with pd.HDFStore('example.h5') as store:
    store.append('trades',
                 df,
                 format='t',
                 data_columns=['side', 'stock'],    # indexed for queries
                 min_itemsize={'stock': 8})          # max 8 bytes per ticker

# --- Read everything back ---
with pd.HDFStore('example.h5', mode='r') as store:
    all_trades = store['trades']                    # or store.select('trades')

# --- Server-side query: buy side only ---
with pd.HDFStore('example.h5', mode='r') as store:
    buys = store.select('trades', where='side == 1')
```

Three points are worth noting. `store.append` can be called in a loop: each call adds rows to the existing table. Queries in `where` use PyTables syntax and are evaluated inside the C library, without loading the full table into Python. And `timestamp` stored as `timedelta64[ns]` preserves nanosecond precision and supports arithmetic without any conversion.

7 NASDAQ ITCH 5.0 Schema

NASDAQ TotalView-ITCH 5.0 is a binary, sequenced, multicast feed of every order-book event on the NASDAQ exchange — add, modify, cancel, execute, and cross. A single trading day exceeds 200 million messages and about 10 GB uncompressed.

One HDF5 table per message type
The parser writes each ITCH message type to its own key in `itch.h5`: `/A` (add order), `/D` (delete), `/E` (execute), `/U` (replace), `/P` (non-cross trade), `/Q` (cross trade), `/I` (imbalance indicator), etc. Separating by message type preserves the original schema and avoids mixing fields across incompatible event kinds.

Timestamps as `timedelta64[ns]`
The 6-byte nanosecond field is unpacked from big-endian bytes via `int.from_bytes` and converted with `pd.to_timedelta`. Storing as `timedelta64` (time elapsed since midnight) enables direct nanosecond-precision arithmetic and range queries with no secondary conversion step.

Alpha fields decoded and integer-encoded
Single-byte categorical fields arrive as raw ASCII bytes from `struct.unpack`. They are first decoded with `str.decode('utf-8')` and stripped, then mapped to integers per the encoding tables in the next section. Integer storage eliminates the need for `min_itemsize`, reduces column width, and makes downstream arithmetic (e.g. signed imbalance) natural.

`stock_locate` as indexed column
Each ITCH message carries a 2-byte integer stock locator assigned by NASDAQ. Indexing `stock_locate` as a `data_column` enables `store.select('A', where='stock_locate == 7421')` to retrieve all add-order events for a single stock without a full table scan.

Incremental writes every 25 million messages
The parser accumulates messages in a `defaultdict(list)` and calls `store.append` after every 25 million records, then clears the buffer. This bounds peak RAM usage to roughly 2–4 GB while building a single append-only day file.

8 Categorical Encoding Reference

Column	Raw ASCII	Stored integer	Meaning
buy_sell_indicator	B	+1	Buy order
	S	−1	Sell order
imbalance_direction	B	+1	Buy-side imbalance
	S	−1	Sell-side imbalance
	N	0	No imbalance
	O	0	Insufficient orders (no direction, zero shares)
cross_type	O	0	Opening cross
	C	+1	Closing cross
	H	+2	IPO / halt cross
primary_market_maker	Y / N	1 / 0	Primary MM flag
printable	Y / N	1 / 0	Trade prints to tape

HDF5 key	ITCH message type	Key fields
/S	System event	event_code
/R	Stock directory	stock, market_category, round_lot_size
/H	Trading action	stock_locate, trading_state, reason
/A	Add order	order_ref_num, buy_sell_indicator, shares, price
/F	Add order (MPID)	Same as /A plus attribution
/E	Order executed	order_ref_num, executed_shares, match_number
/C	Executed with price	order_ref_num, execution_price, printable
/X	Order cancel	order_ref_num, cancelled_shares
/D	Order delete	order_ref_num
/U	Order replace	original_ref_num, new_ref_num, shares, price
/P	Non-cross trade	order_ref_num, buy_sell_indicator, shares, price
/Q	Cross trade	shares, cross_price, cross_type
/I	Net order imbalance	imbalance_shares, imbalance_direction, far_price, near_price
/B	Broken trade	match_number
/Y	Reg SHO restriction	stock_locate, reg_sho_action